UNIT V – C++ DEBUGGING SOLUTIONS

Virtual Functions, Polymorphism & Exception Handling

Solutions for Section A: Debugging Questions (5 Problems). Each solution includes the bug location/classification, explanation, corrected runnable code, dry run, and expected terminal output.

Question 1: Non-Virtual Destructor in Polymorphic Deletion

Bug location: Shape::~Shape() is not virtual (the source labels this as Line 7); delete shapePtr; deletes a ComplexPolygon through a Shape* (Line 41). Bug classification: polymorphic deletion through a base pointer with a non-virtual destructor; this is undefined behavior.

Why it happens: render() is virtual, but the destructor is not. The destructor call is therefore not dynamically dispatched through the virtual mechanism. A delete expression whose static type is Shape* cannot safely destroy the derived object unless Shape has a virtual destructor. The derived heap array may not be released.

Correction: declare the base destructor virtual. The virtual destructor entry enables dynamic dispatch to ComplexPolygon::~ComplexPolygon(), which runs first, followed by Shape::~Shape().

Corrected C++ code

```
#include <iostream>

class Shape {
public:
    Shape() { std::cout << "Shape created." << std::endl; }
    virtual ~Shape() {
        std::cout << "Shape destroyed." << std::endl;
    }
    virtual void render() const {
        std::cout << "Rendering generic shape." << std::endl;
    }
};

class ComplexPolygon : public Shape {
private:
    double* vertexArray;
    int vertexCount;
public:
    ComplexPolygon(int count) : vertexCount(count) {
        vertexArray = new double[vertexCount * 2];
        std::cout << "Allocated " << vertexCount
                  << " vertices on heap." << std::endl;
    }
    ~ComplexPolygon() override {
        std::cout << "Deallocating polygon vertex heap memory." << std::endl;
        delete[] vertexArray;
    }
    void render() const override {
        std::cout << "Rendering polygon with " << vertexCount
                  << " vertices." << std::endl;
    }
};

int main() {
    Shape* shapePtr = new ComplexPolygon(1000);
```

```
    shapePtr->render();
    delete shapePtr;
    return 0;
}
```

Dry run and output

Construction: Shape constructor → ComplexPolygon allocates the vertex array. Calling render through Shape* dynamically dispatches to ComplexPolygon::render. Deletion: ComplexPolygon destructor releases the array → Shape destructor runs.

```
Shape created.
Allocated 1000 vertices on heap.
Rendering polygon with 1000 vertices.
Deallocating polygon vertex heap memory.
Shape destroyed.
```

Question 2: Inverted Exception Catch Handlers

Bug location: the catch (const BankingException&) handler is placed before catch (const InsufficientFundsException&). Bug classification: catch-handler ordering; the derived handler is shadowed/unreachable and compilers commonly warn that the exception will be caught by the earlier handler.

C++ checks catch handlers in order. A derived exception object can bind to a handler taking a reference to its base class, so the first BankingException handler catches InsufficientFundsException too. Put the more specific derived handler first, then the base handler, and put catch(...) last.

Corrected C++ code

```
#include <iostream>
#include <string>

class BankingException {
protected:
    std::string msg;
public:
    explicit BankingException(std::string m) : msg(m) {}
    virtual std::string getMessage() const { return msg; }
    virtual ~BankingException() = default;
};

class InsufficientFundsException : public BankingException {
private:
    double shortfall;
public:
    InsufficientFundsException(std::string m, double sf)
        : BankingException(m), shortfall(sf) {}
    double getShortfall() const { return shortfall; }
};

void processWithdrawal(double balance, double amount) {
    if (amount > balance)
        throw InsufficientFundsException("Account overdrawn!", amount - balance);
}

int main() {
    try {
        processWithdrawal(500.0, 750.0);
    }
    catch (const InsufficientFundsException& ife) {
```

```
        std::cout << "Shortfall of: $" << ife.getShortfall() << std::endl;
    }
    catch (const BankingException& be) {
        std::cout << "General Banking Error: " << be.getMessage() << std::endl;
    }
    catch (...) {
        std::cout << "Unknown banking error." << std::endl;
    }
    return 0;
}
```

Dry run and output

750 exceeds the balance 500, so the function throws InsufficientFundsException with shortfall 250. The first, specific handler catches it.

```
Shortfall of: $250
```

Question 3: Exception Rethrowing and Slicing

Bug location: throw e; inside intermediateAuditor(). Bug classification: exception slicing during rethrow. The catch parameter is a DatabaseError reference; throw e; constructs and throws a new exception copy whose static type is DatabaseError, losing TimeoutError-specific data.

Correction: use bare throw; inside the active catch block. It rethrows the original exception object and preserves its dynamic type and derived members.

Corrected C++ code

```
#include <iostream>
#include <exception>
#include <string>

class DatabaseError : public std::exception {
protected:
    std::string errorDetails;
public:
    explicit DatabaseError(std::string msg) : errorDetails(msg) {}
    const char* what() const noexcept override {
        return errorDetails.c_str();
    }
};

class TimeoutError : public DatabaseError {
public:
    int timeoutMs;
    TimeoutError(std::string msg, int ms)
        : DatabaseError(msg), timeoutMs(ms) {}
};

void executeQuery() {
    throw TimeoutError("Query timed out connecting to shard 3", 5000);
}

void intermediateAuditor() {
    try {
        executeQuery();
    }
    catch (const DatabaseError& e) {
```

```
        std::cout << "[AUDIT LOG]: Intercepted DB error." << std::endl;
        throw; // rethrow original exception object
    }
}

int main() {
    try {
        intermediateAuditor();
    }
    catch (const TimeoutError& te) {
        std::cout << "Client caught TimeoutError! Duration: "
                  << te.timeoutMs << " ms" << std::endl;
    }
    catch (const DatabaseError& de) {
        std::cout << "Client caught generic DatabaseError: "
                  << de.what() << std::endl;
    }
    return 0;
}
```

Dry run and output

executeQuery throws TimeoutError. The auditor catches it through a DatabaseError reference and prints the audit message. Bare throw rethrows the same TimeoutError, so main selects the TimeoutError handler.

```
[AUDIT LOG]: Intercepted DB error.
Client caught TimeoutError! Duration: 5000 ms
```

Question 4: Incomplete Abstract Class

Bug location: VideoPacket does not implement getPacketSize(), which remains pure virtual from NetworkPacket. Bug classification: attempted instantiation of an abstract class. The compiler reports an error similar to: “cannot declare variable ‘packet’ to be of abstract type ‘VideoPacket’”; it will also identify getPacketSize() as the unimplemented pure virtual function.

A class is abstract if it has at least one pure virtual function that has not been overridden with a concrete implementation. An abstract class cannot be instantiated directly. A derived class becomes concrete only after implementing every inherited pure virtual function.

Corrected C++ code

```
#include <iostream>

class NetworkPacket {
public:
    virtual ~NetworkPacket() = default;
    virtual void serialize() = 0;
    virtual int getPacketSize() = 0;
};

class VideoPacket : public NetworkPacket {
private:
    int frameID;
public:
    explicit VideoPacket(int id) : frameID(id) {}
    void serialize() override {
        std::cout << "Serializing H.264 Video Frame: "
                  << frameID << std::endl;
    }
```

```
    int getPacketSize() override {
        return 1024; // example packet size in bytes
    }
};

int main() {
    VideoPacket packet(1042);
    packet.serialize();
    std::cout << "Packet size: " << packet.getPacketSize()
              << " bytes" << std::endl;
    return 0;
}
```

Dry run and output

VideoPacket now overrides both serialize() and getPacketSize(), so it is concrete and can be instantiated. The size value 1024 is an illustrative implementation because the question does not specify a required size calculation.

```
Serializing H.264 Video Frame: 1042
Packet size: 1024 bytes
```

Question 5: Pass-by-Value Object Slicing

Bug location: void triggerEncounter(Entity e) takes the polymorphic object by value. Bug classification: object slicing. Passing Dragon as an Entity value copies only the Entity base subobject into e; the copied object is an Entity, so its virtual call dispatches to Entity::attack().

The original Dragon has a vptr associated with its dynamic type. In the sliced by-value parameter, the resulting Entity object has Entity's dynamic type/vptr; Dragon's override and derived portion are not retained. Passing by reference (const Entity&) preserves the original object and its dynamic type for virtual dispatch.

Corrected C++ code

```
#include <iostream>

class Entity {
public:
    virtual ~Entity() = default;
    virtual void attack() const {
        std::cout << "Entity performs generic scratch." << std::endl;
    }
};

class Dragon : public Entity {
public:
    void attack() const override {
        std::cout << "Dragon breathes scorching fire!" << std::endl;
    }
};

void triggerEncounter(const Entity& e) {
    e.attack();
}

int main() {
    Dragon boss;
    triggerEncounter(boss);
    return 0;
}
```

Dry run and output

boss is a Dragon. triggerEncounter receives a reference to the same object, not a sliced copy. The virtual attack() call dynamically dispatches to Dragon::attack().

```
Dragon breathes scorching fire!
```

Quick Revision: Core Fixes

- Polymorphic base class deletion: virtual destructor.
- Catch ordering: derived/specific handlers before base/general handlers; catch(...) last.
- Rethrowing from a catch: bare throw; preserves the original dynamic exception type.
- Abstract class: implement every inherited pure virtual function before instantiation.
- Runtime polymorphism: pass objects by reference or pointer, not by value.